

Other Graph Algorithms and Proofs

This chapter is an overview of other graph algorithms, proofs, and concepts that are important to know. We will first cover adjacency matrices, then Prim's algorithm for minimum spanning trees, isomorphism, and topological sorting for directed acyclic graphs (DAGs).

1.1 Adjacency Matrices

Recall that we can represent graphs as lists of data called adjacency lists. In code, this could look like the following:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

This represents the following graph

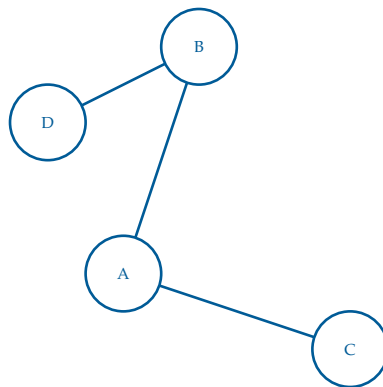


Figure 1.1: A simple (undirected unweighted) graph used to make an adjacency list and adjacency matrix.

We could also represent the same data as a *matrix*, which we have covered in linear algebra. The *adjacency matrix* is 2D array (as compared to an adjacency list, which consists of

multiple 1D arrays) where $\text{matrix}[i][j]$ indicates whether an edge exists from vertex i to vertex j . Commonly, $\text{matrix}[i][j] = 1$ if and only if the edge from vertex i to vertex j exists, else $\text{matrix}[i][j] = 0$. The above graph in Figure 1.1 can be represented as:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

For *undirected* graphs, the adjacency matrix is symmetric, meaning that the transpose of matrix is the same matrix!

$$A = A^T$$

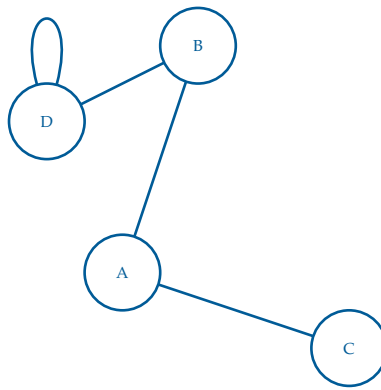


Figure 1.2: A simple graph with an added loop on vertex D.

By convention, a diagonal entry such as $\text{matrix}[k][k]$, where $(k \in \mathbb{Z})$, is non-zero only if there is a loop from a vertex to itself. The value of the entry represents the number of such loops. In this case, there are two edges from vertex (D) to itself in Figure 1.2, so the corresponding diagonal entry is (2).

For *directed* graphs, the adjacency matrix appears a bit different. A *directed adjacency matrix* represents a directed graph, where edges have a direction (arrows). When $A_{ij} = 1$, then an edge exists from vertex i to vertex j .

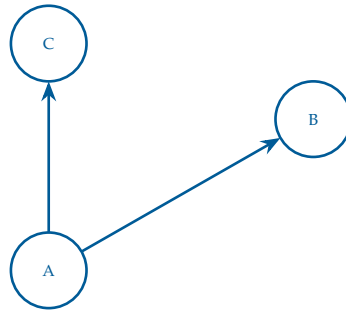


Figure 1.3: A simple directed graph used to make an adjacency matrix.

The resulting adjacency matrix is

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Notice how, now, $A \neq A^T$, because the edges are directional.

Exercise 1 Adjacency Matrix Exercise

	Working Space	
	Answer on Page 5	

1.2 Prim's Algorithm and Kruskal's Algorithm for Minimum Spanning Trees

Prim's Algorithm is minimum spanning tree

FIXME <https://github.com/KontinuaFoundation/sequence/issues/315>

1.3 Travelling Salesperson Problem and Nearest Neighbour

1.4 Isomorphism

1.5 Topological Sort and DAGs

1.6 Proofs

This is a draft chapter from the Kontinua Project. Please see our website (<https://kontinua.org/>) for more details.

Answers to Exercises

Answer to Exercise 1 (on page 3)



INDEX

adjacency matrix, [1](#)

directed adjacency matrix, [2](#)

matrix, [1](#)